BestCV — Interview Prep

Grounded in `github.com/shradha-khapra/ai-resume-ats` (cloned + read directly, incl. notebook outputs).

---

1. Architecture Summary (real data flow)

Frontend (Streamlit) → Auth → Backend (FastAPI) → LLM extraction → NLP scoring → response → async persistence → optional PDF.

1. **Sign-in**: `frontend/streamlit_app.py` handles Supabase email/password + Google OAuth, storing a JWT `access_token` in session state.
2. **Upload**: In the Scorer view, user uploads a PDF/DOCX + optional pasted JD. `frontend/services/api_client.py` POSTs multipart form data + Bearer token to `POST /api/v1/analyze-resume`.
3. **Auth check**: `backend/api/auth.py::get_current_user` verifies the JWT — RS256/ES256 via Supabase's JWKS endpoint, or HS256 via a shared secret — and pulls `user_id` from the `sub` claim server-side.
4. **File parsing**: `backend/services/resume_parser.py` validates MIME type (`python-magic`), extracts text with `pdfplumber` (falls back to `PyPDF2` via a generic `with_fallback` helper), and separately pulls hyperlinks from PDF annotations / DOCX relationships (so GitHub/LinkedIn links survive even if not in visible text).
5. **Orchestration** — `backend/services/resume_analyzer.py::analyze_full_resume()`:
   - **Groq extraction**: `groq_parser.parse_resume()` sends raw text to `llama-3.3-70b-versatile` (temp=0.0) with a JSON-only structured prompt → name, skills, experience, education, projects, action_verbs, keywords. Missing/malformed fields get defaulted in `_validate_resume_result`.
   - **Skill validation**: `ats_scorer.validate_skills_with_projects()` — per skill, substring check against project/experience text first (fast path); if that misses, cosine similarity via the shared `SentenceTransformer` (threshold 0.6).
   - **JD comparison** (if JD pasted): a second Groq call parses the JD into required/preferred skills + keywords; `jd_matcher.py` computes fuzzy keyword overlap (`rapidfuzz` + a skill-alias dictionary) and semantic similarity (cosine, on the first 5,000 chars of each text).
   - **Scoring**: `ats_scorer.calculate_overall_score()` combines 5 weighted components — formatting (20), keywords (25), content (25), skill_validation (15), ats_compatibility (15) — re-weighted into a composite (skills+keywords 40%, content 30%, formatting 15%, ats_compatibility 15%), then applies bonuses (perfect grammar, high skill validation) and penalties (missing JD keywords).
   - **Issue detection**: `feedback_engine.analyze_issues()` runs ~10 independent rule checks (missing sections, weak bullet openers, no metrics, incomplete contact info, poor formatting) → structured `IssueDetail` objects with severity, fix instructions, and a rewritten example.
6. **Response**: assembled into a Pydantic `AnalysisResponse`, rendered by Streamlit components (dashboard, score breakdown, skill validation, JD comparison).
7. **Persistence**: `backend/database/supabase_db.py::save_analysis()` fires *after* the response is built, via raw `httpx` calls to Supabase's PostgREST REST API using the service-role key. Failure here is caught and logged — non-blocking.

8. **PDF export** (on demand): `report_generator.py` renders 4 Jinja2 HTML templates from the stored analysis JSON; `pdf_export.py` converts each to a WeasyPrint `Document` and merges all pages into one PDF, streamed back as a download.
9. **Offline, not in the request path**: a 3-notebook pipeline (EDA/cleaning → base embeddings → fine-tuning) builds and evaluates the fine-tuned Sentence-Transformer described in resume bullet #2. This is a separate, non-runtime artifact — see the honest gap in §4.

---

2. Bullet-by-Bullet Defense

"Built a full-stack ATS scoring system that parses resumes/JDs via Groq (Llama 3.3-70B) using structured extraction prompts, scoring across 5 weighted categories."

- **Where**: `backend/services/groq_parser.py` (prompts + `GROQ_MODEL='llama-3.3-70b-versatile'`, temp 0.0, one retry with a stricter prompt if JSON parsing fails); `backend/services/ats_scorer.py::calculate_overall_score()` (the 5-category weighting).
- **How to say it**: "I use a JSON-only system prompt at temperature 0 so extraction is deterministic and machine-parseable — no markdown fences, no explanation text. If the first response fails to parse, I retry once with an explicit 'that wasn't valid JSON' instruction. The five categories aren't just averaged — I fold skill-validation into the keyword score first (60/40), then weight that combined signal at 40% of the final score, content at 30%, and formatting/ATS-compatibility at 15% each, with a few bonus/penalty adjustments layered on top for things like perfect grammar or heavy JD-keyword gaps."

"Fine-tuned a Sentence-Transformer (mpnet-base-v2) on a custom 266-pair resume-JD dataset via CosineSimilarityLoss, cutting match-score MAE by 70% (0.156 → 0.047) vs. the base model."

- **Where**: entirely in `jupyter notebooks/03_BERT_FINETUNEipynb.ipynb`. Confirmed from actual saved cell outputs: 266 pairs loaded → stratified 70/15/15 split (186/40/40) → `InputExample(texts=[resume, jd], label=match_score)` → base `all-mpnet-base-v2` baseline MAE **0.1562** → fine-tuned with `losses.CosineSimilarityLoss`, batch size 16, 10 epochs, 10% warmup, `EmbeddingSimilarityEvaluator` on the val set, `save_best_model=True` → test MAE **0.0468** (70.0% reduction). These numbers are real and reproducible, not invented.
- **How to say it**: "The dataset had continuous match scores (0–1), not just similar/dissimilar pairs, so `CosineSimilarityLoss` was the right fit — it directly regresses the embedding cosine similarity toward that continuous label, the same setup sentence-transformers uses for STS-style benchmarks. I held out a genuine 40-pair test set the model never saw during training or validation, and used `save_best_model` so I evaluated the best validation checkpoint, not just whatever the last epoch happened to produce."
- **Be ready for**: "Is this model actually being called by the live app?" — see §4, be honest here.

"Engineered an embedding-based skill-validation pipeline (cosine similarity) to verify claimed skills against real project/experience evidence, beyond keyword matching."

- **Where**: `backend/services/ats_scorer.py::validate_skills_with_projects()` (lines ~103–164), helpers `_skill_matches()` and `_calculate_semantic_similarity()`.
- **How to say it**: "For each Groq-extracted skill, I first do an O(1) case-insensitive substring check against the concatenated project/experience text — catches the common case where the skill is literally named.

Only if that misses do I embed the skill string and the project text separately and compute cosine similarity, accepting matches at ≥0.6. That's what lets 'built REST APIs' validate a claimed skill like 'API design' even without the literal word 'API design' appearing — pure keyword matching would miss that."

- **Caveat**: this is the one piece of the embedding story that *is* live in production — but it runs on the same currently-deployed base model as everything else, not the fine-tuned one (see §4).

"Built a rule-based recommendation engine prioritizing fixes by severity and impact, with PDF export and Supabase-backed history."

- **Where (this one has a real nuance)**: there are *two* recommendation implementations in the repo.
  - `backend/services/recommendation_engine.py` — a `Priority` enum + numeric `impact_score` per recommendation, with `_prioritize_recommendations()` sorting by `(priority tier, -impact_score)`. This matches the bullet's wording closely — **but it's never imported anywhere outside its own file.**
  - `backend/services/feedback_engine.py::analyze_issues()` — the one actually wired into `resume_analyzer.py`. ~10 rule checks, each tagged with a `severity_level` string and `ats_impact`, plus `how_to_fix` / `action_items` / a rewritten `example_improvement`. Not dynamically re-sorted by a numeric score, but `report_generator.py` does bucket issues into high/medium/low groups for the PDF.
- **PDF export**: `report_generator.py` (Jinja2 → HTML) + `pdf_export.py` (WeasyPrint → merged PDF).
- **Supabase history**: `database/supabase_db.py` — raw `httpx` calls to Supabase's REST API with the service-role key, filtered server-side by `user_id`.
- **How to say it, honestly**: "I actually built this twice. The first version scored and numerically sorted recommendations by priority and impact — that's `recommendation_engine.py`. I later built a more detailed per-issue version with concrete fix instructions and rewritten examples, and that's the one that shipped. It still tags severity and buckets by it for the PDF, but it doesn't do the numeric re-ranking the first version does. If I had more time I'd merge the two."

---

3. Likely Interview Questions

1. "Walk me through what happens end-to-end when someone uploads a resume." → Use §1. Emphasize: file parsing → Groq structured extraction → parallel scoring/validation → rule-based issue detection → response → async Supabase save.

2. "Why Groq/Llama 3.3-70B for extraction instead of GPT-4 or Claude?" → "Groq's inference is fast, which matters for a synchronous request/response UX where the user is watching a spinner — and at temperature 0 with a strict JSON-only prompt, a 70B open model is reliable enough for structured extraction; I didn't need frontier reasoning for 'pull the skills list out of this resume.'" (Be honest this is your design reasoning, not a documented benchmark you ran.)

3. "Why cosine similarity with a fixed 0.6 threshold instead of training a classifier for skill validation?" → "No labeled 'is this skill actually demonstrated' dataset exists, so a classifier needs training data I don't have. Cosine similarity is unsupervised, interpretable — I can show the user the actual similarity score — and the substring fast-path handles the easy cases without touching the model at all."

4. "Why CosineSimilarityLoss for fine-tuning instead of a contrastive or triplet loss?" → "My labels were continuous match scores, not binary same/different pairs or anchor-positive-negative triplets —

CosineSimilarityLoss regresses straight toward that continuous target, which is the natural fit."

5. "266 pairs, 40 in the test set — how confident are you in that 70% MAE number?" → Honest answer: "It's a real number from a genuine held-out set the model never saw. But 40 test pairs is small, so there's real variance I haven't quantified — with more data I'd want k-fold cross-validation before I'd put that number in a paper. For a portfolio project it's a legitimate, reproducible result; I wouldn't claim it's a tight confidence interval."

6. "How did you land on the specific score weights — 40/30/15/15, or 20/25/25/15/15 per category?" → Honest answer: "Hand-tuned based on domain intuition about what recruiters and ATS parsers actually weigh, not learned from labeled outcome data. If I had data on which resumes actually got interviews, I'd want to calibrate these empirically instead."

7. "What happens to the semantic-similarity calculation for a very long resume?" → `jd_matcher.calculate_semantic_similarity` truncates both texts to `[:5000]` characters before encoding. "It's a blunt length guard to bound embedding cost, not sentence-aware — a long two-page resume could lose its most recent role from the similarity signal. I'd switch to a smarter truncation (e.g. keep the most recent section) or chunk-and-average if this became a real issue."

8. "How do you prevent one user from seeing another user's history?" → "`get_current_user` verifies the JWT server-side and takes `user_id` from the token's `sub` claim — never from a client-supplied parameter — then every history/delete query filters by that `user_id`." Good follow-up to volunteer: "That said, I'm using the Supabase service-role key for these REST calls, which bypasses Postgres RLS — so all the access control lives in my application code, not defense-in-depth at the database layer. Row-level security policies on the anon key would be the more robust setup."

9. "Resumes contain PII, and you're sending full resume text to a third-party LLM API — any concerns there?" → "Yes — name, email, phone, sometimes home address all go to Groq as part of the extraction prompt. I do have a location/address detector built with spaCy NER + regex for the privacy-risk part of the ATS-compatibility score, but it isn't a pre-Groq redaction step — it currently runs (or would run — see the gap below) against the parsed resume for scoring, not to scrub the LLM input. For a production version I'd want to redact PII before the LLM call, not just flag it afterward."

10. "What's the first thing that breaks if this had to handle real concurrent load?" → "The FastAPI endpoint is `async def`, but both the Groq SDK call and the SentenceTransformer `.encode()` calls are synchronous, blocking calls made without offloading to a thread pool. Under concurrency, those block the event loop, so requests queue up rather than actually running in parallel — that's the first bottleneck I'd fix, either with `asyncio.to_thread` or an async Groq client."

11. "Your resume-parsing retry logic — walk me through what happens if Groq returns malformed JSON." → Describe the intended design (retry once with a stricter prompt). If pushed on the actual code, see §4 — there's a real bug here worth knowing about rather than being surprised by.

12. "If you had another month on this, what would you build next?" → Good spot to mention: wiring the fine-tuned model into production, merging the two recommendation engines, moving blocking LLM/embedding calls off the event loop, and pre-LLM PII redaction.

---

4. Weak Spots / Honest Gaps

Flagging these so you're not caught flat-footed — not all of them need to be volunteered, but you should never look surprised if asked directly.

- **The fine-tuned model isn't wired into the live app (biggest one).** `backend/core/config.py` defaults `SENTENCE_TRANSFORMER_MODEL` to `"all-MiniLM-L6-v2"` (confirmed by the README's tech stack section too), and nothing in the backend ever loads the notebook's saved checkpoint. The skill-validation and JD-similarity code paths both run on the stock base model today. → *If asked*: "The fine-tuning was validated end-to-end in a notebook and the result is real, but I hadn't finished plumbing the checkpoint into the deployed API before — it's a config change (point `SENTENCE_TRANSFORMER_MODEL` at the saved model directory) plus packaging the weights, not a redesign."

- **`recommendation_engine.py` is dead code.** Fully built (Priority enum, numeric impact scoring, sorting), but not imported anywhere outside itself — confirmed via repo-wide grep. → *If asked*: "I built two versions and shipped the more detailed one; the numeric-priority version didn't make it into the final wiring."

- **Inverted condition in the Groq retry logic**, `backend/services/groq_parser.py::parse_resume()` (~lines 111–135): the check is `if result is None: return _validate_resume_result(result)` — backwards. On a *successful* first parse it skips the return and needlessly calls Groq a second time every request (extra cost/latency); on a genuinely *failed* first parse it calls `_validate_resume_result(None)`, which crashes with a `TypeError` instead of retrying. (`parse_job_description()` right below it has the correct, non-inverted version — good reference for what it should look like.) → *If asked*: "Good catch — that condition is inverted. It's the first thing I'd fix; right now it either double-calls Groq on success or crashes instead of retrying on failure."

- **Grammar checking and location/privacy detection are stubbed at runtime**, even though a real location detector exists. `ats_scorer.py::detect_location_info()` (spaCy NER + address/ZIP regex) is fully implemented but never called; the live pipeline always uses `file_utils.get_default_grammar_results()` / `get_default_location_results()`, which hardcode "0 errors" / "unavailable." → *If asked*: "Those two scoring inputs are implemented but not connected to the main pipeline yet — right now they always report a clean bill of health, so the grammar and privacy deductions never actually fire."

- **`validate_file()` can crash on an empty upload.** `backend/services/resume_parser.py` line ~41 returns a 2-tuple (`return False, 'uploaded file is empty...'`) on the empty-file branch, but the function signature and every other branch return a 3-tuple — the caller unpacks 3 values, so a 0-byte upload raises `ValueError: not enough values to unpack` instead of showing the friendly message. → *If asked*: "An edge case I hadn't tested — empty file uploads hit an unpacking error instead of the intended validation message."

- **Config-declared score weights are unused.** `SCORE_WEIGHTS`, `JD_KEYWORD_WEIGHT`, `JD_SEMANTIC_WEIGHT` are defined in `core/config.py` but never imported; the actual weights are hardcoded separately (and currently, coincidentally, identically) inside `ats_scorer.py` and `jd_matcher.py`. → *If asked*: "Those config values are aspirational right now — changing them wouldn't change scoring behavior. I'd refactor the scorer to actually import them."

- **`.doc` upload is accepted by validation but always fails extraction.** `extract_text_from_doc()` unconditionally raises "Legacy .doc format is not supported," even though `.doc`'s MIME type is in the

supported list. → *If asked*: "Scaffolded but not implemented — I'd either add a converter or drop `.doc` from the accepted types."

- **Blocking LLM/embedding calls inside an async endpoint** — see Q10 above. Real scalability limit, not yet load-tested.

---

5. "Tell Me About This Project" — Two Lengths

30 seconds: "BestCV is a full-stack ATS resume scorer — Streamlit frontend, FastAPI backend. Someone uploads a resume and optionally a job description; I use Llama 3.3-70B on Groq to extract structured data — skills, experience, projects — from the raw text, then score the resume across five weighted categories. The part I'm most proud of is the skill-validation step: instead of just keyword matching, I use sentence-embedding cosine similarity to check whether a claimed skill actually shows up in the person's projects or experience. I also fine-tuned a Sentence-Transformer on a 266-pair resume/JD dataset, cutting match-score error by 70% versus the base model. Results save to Supabase and export as a PDF report."

2 minutes: "BestCV scores how well a resume matches a job description and gives actionable feedback — Streamlit frontend, FastAPI backend, Supabase for auth and history.

The pipeline: a user uploads a PDF or DOCX and optionally pastes a JD. I extract the raw text, then send it to Groq's Llama 3.3-70B with a structured, JSON-only prompt to pull out skills, experience, education, and projects — deterministically, at temperature 0. From there I do two things beyond simple keyword matching. First, skill validation: for every skill the person claims, I check whether it's actually demonstrated in their projects or experience, using embedding cosine similarity as a fallback when there's no exact keyword match. Second, if there's a JD, I compute semantic similarity between the full resume and JD text the same way. All of that feeds a weighted scoring function across five categories — formatting, keywords, content quality, skill validation, and ATS compatibility — plus a rule-based feedback engine that flags specific issues, like a missing Projects section or bullet points with no quantified impact, each with a concrete rewrite suggestion.

Separately, I fine-tuned a Sentence-Transformer — `all-mpnet-base-v2` — on a custom 266-pair resume/JD dataset I built and cleaned myself, using CosineSimilarityLoss since my labels were continuous match scores rather than discrete pairs. That cut test-set MAE from 0.156 to 0.047 against the un-tuned base model, evaluated on a genuine held-out test set.

Results get saved to Supabase per-user and can be exported as a combined PDF report via WeasyPrint. If I kept going, the next things I'd do are get that fine-tuned model actually deployed behind the live API — right now it's validated offline but not yet wired in — and move the LLM/embedding calls off the main event loop so the API holds up better under concurrent load."